[44]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[45]:
```
pervised learning\project 2(robust regression engine)\Advanced_Regression_HousePrice_Dataset_3800 - Advanced_Regression_HousePrice_Dataset_3800.csv.csv")
```

[45]:

	property_id	sale_date	area_sqft	bedrooms	bathrooms	location_score	property_age	distance_city_km	near_school	near_metro	crime_rate_index	house_price_inr
0	200001	2014-01-01	2181	6	4	8.1	21	3.8	0	0	4.84	35154898
1	200002	2019-12-01	2383	5	4	5.3	28	10.9	1	1	2.89	26710893
2	200003	2016-10-01	1047	3	3	5.9	7	27.5	0	1	4.04	11216242
3	200004	2013-03-01	1753	4	3	7.0	27	12.1	0	0	3.28	21984310
4	200005	2013-07-01	1728	4	4	10.0	32	1.4	0	1	3.84	25080429

Part B: Dataset Understanding & Preparation

[46]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3800 entries, 0 to 3799
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   property_id       3800 non-null   int64  
 1   sale_date         3800 non-null   object 
 2   area_sqft         3800 non-null   int64  
 3   bedrooms          3800 non-null   int64  
 4   bathrooms         3800 non-null   int64  
 5   location_score    3800 non-null   float64
 6   property_age      3800 non-null   int64  
 7   distance_city_km  3800 non-null   float64
 8   near_school       3800 non-null   int64  
 9   near_metro        3800 non-null   int64  
 10  crime_rate_index  3800 non-null   float64
 11  house_price_inr   3800 non-null   int64  
dtypes: float64(3), int64(8), object(1)
memory usage: 356.4+ KB
```

[47]:
```
df.shape
```

[47]:
```
(3800, 12)
```

[48]:
```
df.describe()
```

[48]:

	property_id	area_sqft	bedrooms	bathrooms	location_score	property_age	distance_city_km	near_school	near_metro	crime_rate_index	house_price_inr
count	3800.00000	3800.000000	3800.000000	3800.000000	3800.000000	3800.000000	3800.000000	3800.000000	3800.000000	3800.000000	3.800000e+03
mean	201900.50000	1716.925526	3.428158	2.916316	6.502237	22.537105	13.085132	0.548421	0.472895	4.242911	2.071940e+07
std	1097.10984	582.996559	1.356682	1.133540	1.766945	12.325740	6.537425	0.497715	0.499330	2.045371	8.707465e+06
min	200001.00000	500.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.000000	0.000000	0.500000	1.506126e+06
25%	200950.75000	1322.000000	2.000000	2.000000	5.300000	14.000000	8.500000	0.000000	0.000000	2.810000	1.446895e+07
50%	201900.50000	1700.500000	3.000000	3.000000	6.500000	20.000000	13.000000	1.000000	0.000000	4.220000	1.989180e+07
75%	202850.25000	2105.000000	4.000000	4.000000	7.700000	29.000000	17.500000	1.000000	1.000000	5.650000	2.596062e+07
max	203800.00000	3776.000000	7.000000	6.000000	10.000000	80.000000	38.700000	1.000000	1.000000	12.000000	5.930315e+07

[49]:
```
df.isna().sum()
```

[49]:
```
property_id         0
sale_date           0
area_sqft           0
bedrooms            0
bathrooms           0
location_score      0
property_age        0
distance_city_km    0
near_school         0
near_metro          0
crime_rate_index    0
house_price_inr     0
dtype: int64
```

Feature Extraction

[51]:
```
## feature extraction
df["sale_date"] = pd.to_datetime(df["sale_date"])
df["date"] = df["sale_date"].dt.day
df["month"] = df["sale_date"].dt.month
df["year"] = df["sale_date"].dt.year

df.drop("sale_date", axis = 1, inplace=True)
```

Train Test Split

[52]:
```
from sklearn.model_selection import train_test_split
x = df.drop("house_price_inr", axis=1)
y = df["house_price_inr"]
```

[53]:
```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.3,
    random_state=32
)
```

Scaling

[54]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Part C: Regularized Linear Models

[55]:
```
from sklearn.linear_model import RidgeCV, LassoCV

ridge = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
ridge.fit(x_train_scaled, y_train)
print("Ridge best alpha:", ridge.alpha_)

lasso = LassoCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
lasso.fit(x_train_scaled, y_train)
print("Lasso best alpha:", lasso.alpha_)
```

```
Ridge best alpha: 1.0
Lasso best alpha: 100.0
```

[56]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

ridge_train_pred = ridge.predict(x_train_scaled)
ridge_test_pred = ridge.predict(scaler.transform(x_test))

lasso_train_pred = lasso.predict(x_train_scaled)
lasso_test_pred = lasso.predict(scaler.transform(x_test))
```

```
print("Ridge Regression:-")
print("Training RMSE:", np.sqrt(mean_squared_error(y_train, ridge_train_pred)))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, ridge_test_pred)))
print("r2_score:", r2_score(y_test, ridge_test_pred))


print("\nLasso Regression:-")
print("Training RMSE:", np.sqrt(mean_squared_error(y_train, lasso_train_pred)))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, lasso_test_pred)))
print("R2_score:", r2_score(y_test, lasso_test_pred))
```

```
Ridge Regression:-
Training RMSE: 2513950.8536799694
Test RMSE: 2444885.596068199
r2_score: 0.918839985041814

Lasso Regression:-
Training RMSE: 2513945.880619565
Test RMSE: 2445027.2036317997
R2_score: 0.9188305832073628
```

[57]:
```
coefficient = pd.DataFrame({
    "Features":x_train.columns,
    "Ridge_coefficient":ridge.coef_,
    "Lasso_coefficient":lasso.coef_
})

print(coefficient.round(3))
print("Ridge zero coefficient:", np.sum(np.isclose(ridge.coef_, 0)))
print("Lasso zero coefficient:", np.sum(np.isclose(lasso.coef_, 0)))
```

```
           Features  Ridge_coefficient  Lasso_coefficient
0       property_id         -10546.365         -10460.773
1         area_sqft        7072501.905        7080902.243
2          bedrooms         329313.471         322539.463
3         bathrooms         189230.145         188942.336
4    location_score        3724825.020        3727192.492
5      property_age        -663281.218        -663354.143
6   distance_city_km         -21591.360         -19958.541
7       near_school         -18504.017         -18423.288
8        near_metro          34533.043          34577.655
9   crime_rate_index        -198659.656        -198498.421
10              date              0.000              0.000
11             month           4172.270           3938.765
12              year         310603.648         310660.051
Ridge zero coefficient: 1
Lasso zero coefficient: 1
```

Part D: Cross-Validation Strategies

[58]:
```
from sklearn.model_selection import KFold, StratifiedKFold, LeaveOneOut, TimeSeriesSplit, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
```

[59]:
```
model = make_pipeline(StandardScaler(), Ridge(alpha=1))
kfold = KFold(n_splits=5, shuffle=True, random_state=32)
scores = cross_val_score(model, x_train, y_train, cv=kfold, scoring="neg_mean_squared_error")
print("k-fold RMSE:", np.sqrt(-scores).mean())
```

```
k-fold RMSE: 2528879.6734521366
```

[60]:
```
y_bins = pd.qcut(y_train, q=5, labels=False)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=32)
scores = cross_val_score(model, x_train, y_train, cv=skf.split(x_train, y_bins), scoring="neg_mean_squared_error")
print("Stratified K-Fold RMSE:", np.sqrt(-scores).mean())
```

```
Stratified K-Fold RMSE: 2522953.353664779
```

[61]:
```
loo = LeaveOneOut()
scores = cross_val_score(model, x_train, y_train, cv=loo, scoring="neg_mean_squared_error", n_jobs=-1)
print("LOOCV RMSE:", np.sqrt(-scores).mean())
```

```
LOOCV RMSE: 1929860.6584261672
```

[62]:
```
x_time = x_train.sort_values(["year", "month", "date"])
y_time = y_train.loc[x_time.index]

tscv = TimeSeriesSplit(n_splits=5)

scores = cross_val_score(model, x_time, y_time,cv=tscv,
                         scoring="neg_mean_squared_error")

print("Time Series RMSE:", np.sqrt(-scores).mean())
```

```
Time Series RMSE: 2521371.4123724075
```

Part E: Tree-Based Regression Models

[74]:
```
from sklearn.tree import DecisionTreeRegressor
dt = DecisionTreeRegressor(random_state=32)
dt.fit(x_train, y_train)

y_pred_dt = dt.predict(x_test)

print("Decision tree RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_dt)))
print("R2_score:", r2_score(y_test, y_pred_dt))
```

```
Decision tree RMSE: 3286658.2956653354
R2_score: 0.8533324144237239
```

[77]:
```
dt_tuned = DecisionTreeRegressor(
    max_depth=10, min_samples_leaf=5, min_samples_split=10, random_state=32
)
dt_tuned.fit(x_train, y_train)
pred_dt_tuned = dt_tuned.predict(x_test)

print("Tuned decision tree RMSE:", np.sqrt(mean_squared_error(y_test, pred_dt_tuned)))
print("R2_score:", r2_score(y_test, y_pred_dt))
```

```
Tuned decision tree RMSE: 2868740.2829214227
R2_score: 0.8533324144237239
```

[79]:
```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(
    n_estimators=100, max_depth=10, random_state=32, n_jobs=-1
)

rf.fit(x_train, y_train)
pred_rf = rf.predict(x_test)

print("Tuned decision tree RMSE:", np.sqrt(mean_squared_error(y_test, pred_rf)))
print("R2_score:", r2_score(y_test, pred_rf))
```

```
Tuned decision tree RMSE: 2311358.053230028
```

```
results = {
    "Ridge": r2_score(y_test, ridge.predict(x_test_scaled)),
    "Decision Tree": r2_score(y_test, y_pred_dt),
    "Random Forest": r2_score(y_test, pred_rf),
    "Linear SVR": r2_score(y_test, y_pred_linear),
    "RBF SVR Tuned": r2_score(y_test, grid.best_estimator_.predict(x_test))
}

print(pd.Series(results, name="R2 Score").sort_values(ascending=False))
```

[92]:
```
print("MSE:", mean_squared_error(y_test, pred_rf))
print("MAE:", mean_absolute_error(y_test, pred_rf))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_rf)))
print("R2 Score:", r2_score(y_test, pred_rf))
```

```
MSE: 5342376050231.305
MAE: 1759979.4454222734
RMSE: 2311358.053230028
R2 Score: 0.9274630168085167
```